

Artificial Intelligence Core Concepts Handout

SUMMER 2026 — BSIT, 3RD SEMESTER

Foundations • Agents • Search • Games • CSP • Planning • Knowledge • Learning •
Ethics

Contents

1	Introduction to Artificial Intelligence	2
1.1	A Brief History	2
1.2	Approaches to AI	2
1.3	Types of AI by Capability	2
2	Intelligent Agents	3
2.1	Types of Agents	3
2.2	Properties of Task Environments (PEAS)	3
3	Problem Solving by Search	3
3.1	Uninformed (Blind) Search	4
3.2	Informed (Heuristic) Search	4
3.3	Search Strategy Comparison	4
3.4	Worked Search Example	5
3.5	Local Search and Optimization	5
4	Adversarial Search and Game Playing	5
4.1	The Minimax Algorithm	5
4.2	Alpha-Beta Pruning	6
5	Constraint Satisfaction Problems (CSP)	6
5.1	Common CSP Examples	6
5.2	Solving Techniques	7
6	Planning in AI	7
6.1	The STRIPS Representation	7
6.2	Forward vs. Backward Planning	8
7	Knowledge Representation and Reasoning	8
7.1	Logic-Based Representation	8

7.2	Rule-Based Expert Systems	8
7.3	Semantic Networks and Frames	8
7.4	Forward Chaining vs. Backward Chaining	9
8	Reasoning Under Uncertainty	9
8.1	Bayesian Networks	9
8.2	Fuzzy Logic	9
9	Machine Learning Fundamentals	10
9.1	Main Categories	10
9.2	The ML Workflow	10
9.3	Core Supervised Learning Algorithms	10
9.4	Core Unsupervised Learning Algorithms	11
9.5	Key Terminology	11
9.6	Model Evaluation Metrics (Core for Exams)	11
9.7	Bias-Variance Tradeoff	11
10	Reinforcement Learning Fundamentals	11
10.1	Markov Decision Process (MDP)	12
10.2	Policies, Value Functions, and Q-Values	12
10.3	Q-Learning	12
11	Neural Networks and Deep Learning	12
11.1	The Perceptron: A Single Neuron	13
11.2	Convolutional Neural Networks (CNNs)	13
11.3	Recurrent Neural Networks (RNNs) and LSTMs	13
12	Natural Language Processing and Computer Vision	14
12.1	Natural Language Processing (NLP)	14
12.2	Computer Vision	14
13	Applications of AI	15
14	Ethical and Responsible AI	15
15	Core Mathematical Foundations for AI	15
15.1	Linear Algebra Essentials	15
15.2	Probability Essentials	16
15.3	Optimization Essentials	16
16	Exam-Oriented Worked Examples	16
17	Quick Revision Glossary	17
18	Question Bank Blueprint (80 Marks Support)	19
18.1	Important Theory Questions	19
18.2	Important Numerical/Analytical Questions	20
18.3	Sample MCQs with Answer Key	20

19 Summary**21**

1 Introduction to Artificial Intelligence

📖 Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science concerned with building systems that can **perceive**, **reason**, **learn**, and **act** to achieve goals, performing tasks that normally require human intelligence.

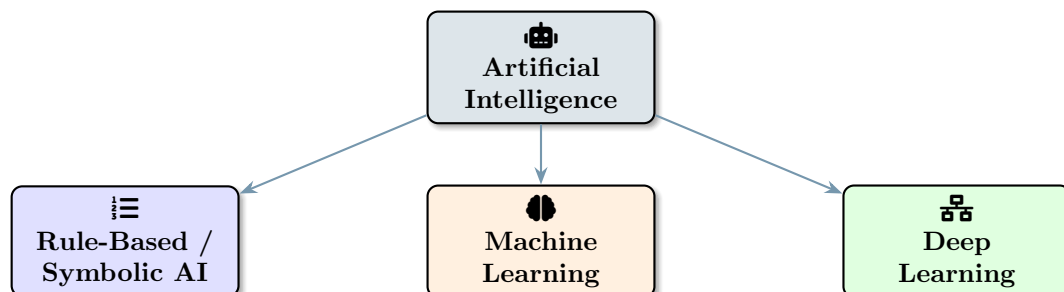
💡 Why AI Matters

AI powers search engines, recommendation systems, voice assistants, self-driving cars, medical diagnosis tools, fraud detection, and chatbots. As a BSIT student, understanding AI fundamentals prepares you to design, use, and evaluate intelligent systems responsibly.

1.1 A Brief History

- **1950:** Alan Turing proposes the *Turing Test* as a measure of machine intelligence.
- **1956:** The term “Artificial Intelligence” is coined at the Dartmouth Conference.
- **1980s:** Expert systems and rule-based AI become popular in industry.
- **1990s–2000s:** Machine learning and statistical approaches gain traction (e.g., IBM Deep Blue defeats a chess champion in 1997).
- **2010s–present:** Deep learning revolution — image recognition, natural language processing, and generative AI (ChatGPT, GPT-4, and beyond).

1.2 Approaches to AI



- **Rule-Based (Symbolic) AI:** Explicit if-then rules and logic written by humans (expert systems).
- **Machine Learning:** Systems learn patterns automatically from data instead of hard-coded rules.
- **Deep Learning:** A subset of ML using multi-layer neural networks for complex tasks like vision and language.

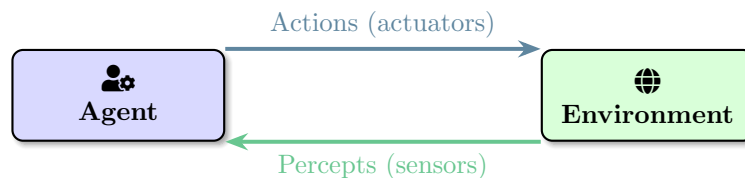
1.3 Types of AI by Capability

- **Narrow AI (Weak AI):** Designed for a specific task (spam filters, recommendation engines, voice assistants). All AI in use today is narrow AI.
- **General AI (Strong AI):** A hypothetical system with human-level intelligence across all tasks. Does not yet exist.
- **Super AI:** A hypothetical AI surpassing human intelligence in all domains. Purely theoretical.

2 Intelligent Agents

Intelligent Agent

An intelligent agent **perceives** its environment through sensors and **acts** upon it through actuators to achieve goals, following an *agent function* that maps percepts to actions.



2.1 Types of Agents

- **Simple Reflex Agents:** Act only on the current percept using condition–action rules (e.g., a thermostat).
- **Model-Based Reflex Agents:** Maintain an internal state to track aspects of the world not directly observed.
- **Goal-Based Agents:** Choose actions that help achieve an explicit goal, considering future consequences.
- **Utility-Based Agents:** Choose actions that maximize a utility function, allowing trade-offs between competing goals.
- **Learning Agents:** Improve performance over time using feedback from a performance element and a learning element.

2.2 Properties of Task Environments (PEAS)

PEAS	Description
Performance measure	Criteria for success (e.g., accuracy, speed, safety).
Environment	The surroundings the agent operates in (e.g., roads, chessboard, web).
Actuators	Mechanisms the agent uses to act (e.g., wheels, screen, robotic arm).
Sensors	Mechanisms the agent uses to perceive (e.g., camera, microphone, keyboard).

3 Problem Solving by Search

Search Problem

A search problem consists of an **initial state**, a set of **actions**, a **transition model**, a **goal test**, and a **path cost** function. Solving it means finding a sequence of actions from the initial state to a goal state.

3.1 Uninformed (Blind) Search

- **Breadth-First Search (BFS):** Explores all nodes at the current depth before moving deeper; guarantees the shallowest solution.
- **Depth-First Search (DFS):** Explores as far as possible along a branch before backtracking; memory-efficient but not always optimal.
- **Uniform Cost Search:** Expands the node with the lowest path cost so far; optimal for weighted graphs.
- **Iterative Deepening DFS (IDDFS):** Repeats DFS with increasing depth limits $(0, 1, 2, \dots)$; combines BFS's completeness with DFS's low memory use.
- **Bidirectional Search:** Simultaneously searches forward from the initial state and backward from the goal, meeting in the middle to cut the effective search depth.

3.2 Informed (Heuristic) Search

Heuristic Function

A heuristic $h(n)$ estimates the cost from node n to the nearest goal, guiding the search toward promising paths without exploring the entire space.

- **Greedy Best-First Search:** Expands the node that appears closest to the goal using $h(n)$ only; fast but not always optimal.
- **A* Search:** Expands the node with the lowest $f(n) = g(n) + h(n)$, where $g(n)$ is the cost so far; optimal when $h(n)$ is admissible (never overestimates).

3.3 Search Strategy Comparison

Algorithm	Complete	Optimal	Memory	Notes
BFS	Yes	Yes (unit step cost)	High	Good for shallow solutions; can explode in memory.
DFS	No (in infinite trees)	No	Low	Fast memory-wise but may get trapped in deep branches.
Uniform Cost	Yes	Yes	High	Best when step costs differ.
Iterative Deepening	Yes	Yes (unit step cost)	Low	Combines BFS's completeness with DFS's memory efficiency.
Bidirectional	Yes (conditions apply)	Yes (conditions apply)	Medium	Needs a well-defined goal and reversible actions; can greatly cut search time.
Greedy Best-First	No	No	Medium	Often fast, but can take wrong shortcuts.

Algorithm	Complete	Optimal	Memory	Notes
A*	Yes	Yes (admissible h)	High	Practical and widely used in pathfinding.

3.4 Worked Search Example

⌘ A* in One Minute

Suppose a robot starts at node S and wants to reach goal G . For each frontier node, A* computes:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost from S to n , and $h(n)$ is estimated cost from n to G . If frontier values are $A : (g = 2, h = 5, f = 7)$, $B : (g = 4, h = 2, f = 6)$, and $C : (g = 3, h = 4, f = 7)$, A* chooses B first because it has the minimum f . This process repeats until the goal is selected.

3.5 Local Search and Optimization

- **Hill Climbing:** Moves to the neighboring state with the best value; can get stuck at local maxima.
- **Simulated Annealing:** Allows occasional “worse” moves to escape local maxima, with randomness decreasing over time.
- **Genetic Algorithms:** Maintain a population of candidate solutions that evolve via selection, crossover, and mutation.

4 Adversarial Search and Game Playing

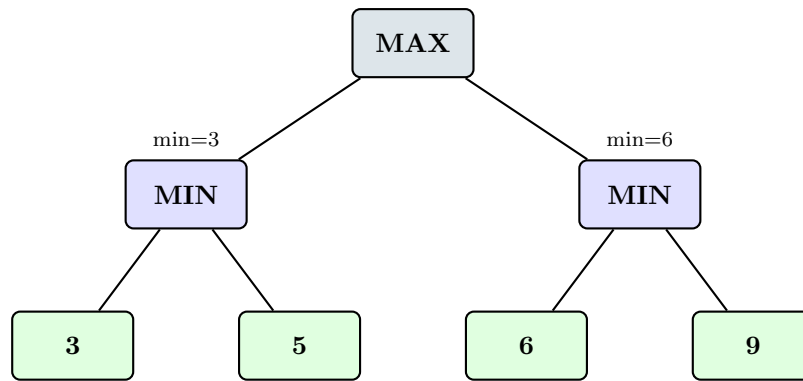
📖 Adversarial Search

Adversarial search handles environments with two or more competing agents, such as chess or tic-tac-toe. Each player chooses moves that best serve its own interest while anticipating the opponent’s best response.

4.1 The Minimax Algorithm

📖 Minimax

Minimax explores the game tree assuming the **MAX** player always picks the move with the highest value and the **MIN** player always picks the move with the lowest value. Values are “backed up” from the leaves toward the root.



4.2 Alpha-Beta Pruning

💡 Alpha-Beta Pruning

Alpha-beta pruning speeds up minimax by skipping branches that cannot change the final decision. It tracks α (best value MAX can guarantee so far) and β (best value MIN can guarantee so far); a branch is pruned whenever $\alpha \geq \beta$.

- Produces the **same decision** as plain minimax, but explores fewer nodes.
- Best case reduces complexity from $O(b^d)$ to $O(b^{d/2})$, where b is the branching factor and d is the depth.
- Exploring the best moves first increases the amount of pruning achieved.

🔗 Worked Example: Minimax Backed-Up Values

Given leaf values 3, 5, 6, 9 (left to right, as in the diagram above):

- Left MIN node: $\min(3, 5) = 3$
- Right MIN node: $\min(6, 9) = 6$
- Root MAX node: $\max(3, 6) = 6$

MAX should choose the move leading to the right subtree, guaranteeing a value of **6**.

5 Constraint Satisfaction Problems (CSP)

📖 Constraint Satisfaction Problem

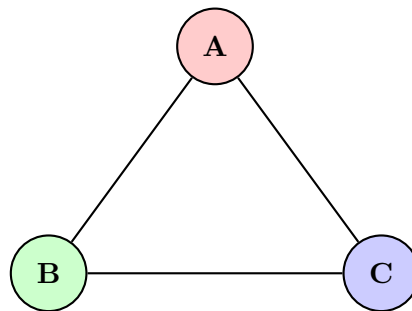
A CSP consists of a set of **variables**, a **domain** of possible values for each variable, and a set of **constraints** restricting which value combinations are allowed. A solution assigns a value to every variable while satisfying all constraints.

5.1 Common CSP Examples

- **Map Coloring:** Color regions so that no two adjacent regions share a color.
- **Sudoku:** Fill digits 1–9 so rows, columns, and boxes have no repeats.
- **N-Queens:** Place N queens on a board so that none attacks another.
- **Timetabling:** Assign time slots to classes without room or teacher conflicts.

5.2 Solving Techniques

- **Backtracking Search:** Assigns variables one at a time and undoes an assignment when it violates a constraint.
- **Forward Checking:** Removes now-invalid values from neighboring domains immediately after each assignment.
- **Constraint Propagation:** Enforces arc consistency so every remaining value stays compatible with related variables.
- **Variable/Value Ordering Heuristics:** Minimum Remaining Values (MRV), Degree Heuristic, and Least Constraining Value guide which variable/value to try next.



Worked Example: Map Coloring

Regions A , B , and C are mutually adjacent; available colors are {Red, Green, Blue}.

- Assign $A = \text{Red}$.
- B is adjacent to A , so $B \neq \text{Red} \Rightarrow$ assign $B = \text{Green}$.
- C is adjacent to both A and B , so $C \neq \text{Red, Green} \Rightarrow$ assign $C = \text{Blue}$.

All constraints are satisfied on the first attempt — no backtracking was needed.

6 Planning in AI

Planning

Planning is the AI task of finding a sequence of actions that transforms an initial state into a goal state, where each action is described by its **preconditions** (what must be true to apply it) and **effects** (what becomes true afterward).

6.1 The STRIPS Representation

STRIPS

STRIPS (STanford Research Institute Problem Solver) represents each planning action using three components:

- **Precondition:** Facts that must hold before the action can be executed.
- **Add List:** Facts that become true after the action.
- **Delete List:** Facts that become false after the action.

6.2 Forward vs. Backward Planning

- **Forward (Progression) Planning:** Searches forward from the initial state, applying actions whose preconditions are satisfied, until the goal is reached.
- **Backward (Regression) Planning:** Searches backward from the goal, selecting actions whose effects satisfy part of the goal, until the initial state is reached.

🔗 Worked Example: A Simple Plan

Goal: *Have a cup of tea*. Action BOILWATER: precondition {Kettle Filled}, effect {Water Boiled}. Action MAKETEA: precondition {Water Boiled}, effect {Tea Ready}.

- Start: {Kettle Filled}
- Apply BOILWATER → {Kettle Filled, Water Boiled}
- Apply MAKETEA → {Kettle Filled, Water Boiled, Tea Ready} — **goal reached**

Plan found: ⟨BOILWATER, MAKETEA⟩.

7 Knowledge Representation and Reasoning

📖 Knowledge Representation

Knowledge representation is how an AI system encodes facts about the world so it can reason, draw conclusions, and make decisions.

7.1 Logic-Based Representation

- **Propositional Logic:** Statements that are either true or false, combined using AND ($\wedge$), OR ($\vee$), NOT ($\neg$), and IMPLIES ($\Rightarrow$).
- **First-Order Logic:** Extends propositional logic with objects, relations, and quantifiers ($\forall$, $\exists$) for richer expressiveness.
- **Inference:** Deriving new facts from known facts using rules such as *Modus Ponens*: from P and $P \Rightarrow Q$, conclude Q .

7.2 Rule-Based Expert Systems

💡 Expert System

An expert system uses a **knowledge base** of if-then rules and an **inference engine** to mimic the decision-making of a human expert in a specific domain (e.g., medical diagnosis, loan approval).

7.3 Semantic Networks and Frames

- **Semantic Network:** A graph of nodes (objects/concepts) connected by labeled edges representing relationships (e.g., “Bird” *-is-a-* “Animal”).
- **Frames:** Structured representations that group an object’s attributes (*slots*) and their values, similar to a record or object in programming.
- **Ontology:** A formal, shared vocabulary of concepts and relationships within a domain, enabling knowledge reuse and reasoning across systems.

7.4 Forward Chaining vs. Backward Chaining

- **Forward Chaining (data-driven):** Starts from known facts and repeatedly applies rules to derive new facts until the goal is reached. Common in monitoring and planning systems.
- **Backward Chaining (goal-driven):** Starts from the goal and works backward, checking which rules could conclude it and recursively verifying their conditions. Common in diagnosis and query-answering systems (e.g., Prolog).

Worked Example: Forward Chaining

Rules: (1) If *it has feathers* $\Rightarrow$ *it is a bird*. (2) If *it is a bird* $\Rightarrow$ *it can fly*.

Fact: *Tweety has feathers*.

Forward chaining: feathers $\Rightarrow$ bird (Rule 1) $\Rightarrow$ can fly (Rule 2). Conclusion: **Tweety can fly**.

8 Reasoning Under Uncertainty

Why Uncertainty?

Real-world knowledge is often incomplete or imprecise. AI systems use probability and fuzzy methods to reason sensibly despite uncertainty, instead of requiring rigid true/false facts.

8.1 Bayesian Networks

Bayesian Network

A Bayesian network is a directed acyclic graph where nodes represent random variables and edges represent probabilistic dependencies. Each node stores a conditional probability table (CPT) given its parents, and Bayes' theorem is used to update beliefs as new evidence arrives.

8.2 Fuzzy Logic

Fuzzy Logic

Fuzzy logic allows truth values between 0 and 1 (partial truth) instead of strict true/false, modeling vague concepts such as “hot”, “fast”, or “tall” using **membership functions**.

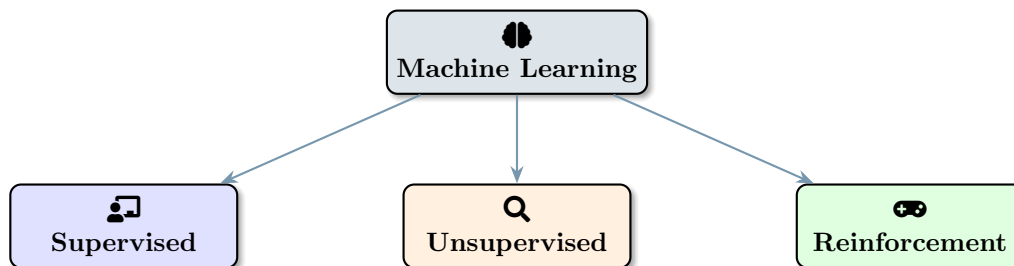
- **Crisp Set:** An element either belongs (1) or does not belong (0) to a set.
- **Fuzzy Set:** An element has a **degree of membership** $\mu(x) \in [0, 1]$ (e.g., a temperature of 28°C might have $\mu_{\text{hot}} = 0.7$).
- **Applications:** Washing machines, air conditioners, anti-lock braking systems, and other control systems requiring smooth decision boundaries.

9 Machine Learning Fundamentals

Machine Learning

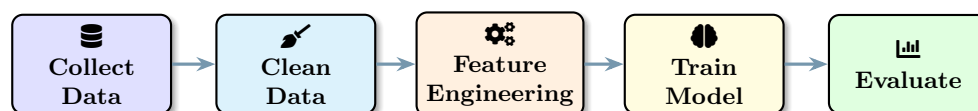
Machine learning is a subset of AI in which systems **learn patterns from data** rather than following explicitly programmed rules, improving performance on a task through experience.

9.1 Main Categories



- **Supervised Learning:** Learns from labeled input-output pairs (classification, regression).
- **Unsupervised Learning:** Finds hidden patterns in unlabeled data (clustering, dimensionality reduction).
- **Reinforcement Learning:** Learns by trial and error, receiving rewards or penalties from the environment.

9.2 The ML Workflow



9.3 Core Supervised Learning Algorithms

- **Linear Regression:** Fits a straight line $\hat{y} = w_0 + w_1x$ to predict continuous values by minimizing mean squared error.
- **Logistic Regression:** Uses the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ to output a probability for binary classification.
- **Decision Trees:** Split data recursively using the feature that gives the highest **information gain** (based on entropy), forming a tree of if-then decisions.
- **K-Nearest Neighbors (KNN):** Classifies a new point by majority vote among its k closest labeled points, using a distance metric (e.g., Euclidean distance).
- **Naïve Bayes:** Applies Bayes' theorem assuming features are conditionally independent given the class:

$$P(\text{class}|\text{features}) \propto P(\text{class}) \prod_i P(\text{feature}_i|\text{class})$$

- **Support Vector Machines (SVM):** Finds the hyperplane that separates classes with the **maximum margin** between the closest points (support vectors).

9.4 Core Unsupervised Learning Algorithms

- **K-Means Clustering:** Partitions data into k clusters by repeating two steps: (1) assign each point to the nearest centroid, (2) recompute each centroid as the mean of its assigned points, until convergence.
- **Hierarchical Clustering:** Builds a tree (dendrogram) of nested clusters by iteratively merging (agglomerative) or splitting (divisive) groups.
- **Principal Component Analysis (PCA):** Projects data onto fewer dimensions (principal components) that capture the maximum variance, used for visualization and noise reduction.

9.5 Key Terminology

Term	Definition
Feature	An input variable describing the data (e.g., age, pixel value).
Label	The target output a supervised model tries to predict.
Training Set	Data used to fit the model.
Test Set	Held-out data used to evaluate the trained model.
Overfitting	When a model memorizes training data but performs poorly on new data.
Accuracy	The proportion of correct predictions out of all predictions.

9.6 Model Evaluation Metrics (Core for Exams)

For binary classification, predictions are summarized using a confusion matrix:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Important formulas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

💡 When Accuracy is Misleading

If 95% of emails are non-spam, a model that always predicts “non-spam” gives 95% accuracy but is useless for detecting spam. In imbalanced datasets, use precision, recall, and F1-score.

9.7 Bias-Variance Tradeoff

- **High Bias (Underfitting):** Model is too simple and misses important patterns.
- **High Variance (Overfitting):** Model is too complex and captures noise.
- **Goal:** Balance model complexity and generalization using validation and regularization.

10 Reinforcement Learning Fundamentals

☰ Reinforcement Learning

Reinforcement Learning (RL) is a learning paradigm where an **agent** interacts with an **environment**, taking actions and receiving **rewards**, aiming to learn a **policy** that maximizes cumulative reward over time.

10.1 Markov Decision Process (MDP)

💡 MDP

An MDP formalizes an RL problem using: a set of **states** S , a set of **actions** A , a **transition function** $P(s'|s, a)$, a **reward function** $R(s, a)$, and a **discount factor** $\gamma \in [0, 1]$ that reduces the value of future rewards.

10.2 Policies, Value Functions, and Q-Values

- **Policy** $\pi(s)$: A strategy that maps states to actions.
- **Value Function** $V(s)$: Expected cumulative reward starting from state s and following policy π .
- **Q-Value** $Q(s, a)$: Expected cumulative reward of taking action a in state s , then following π afterward.

10.3 Q-Learning

☰ Q-Learning Update Rule

Q-learning learns Q-values directly from experience, without needing a model of the environment:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where α is the learning rate, r is the immediate reward, and $\gamma \max_{a'} Q(s', a')$ estimates the best future value.

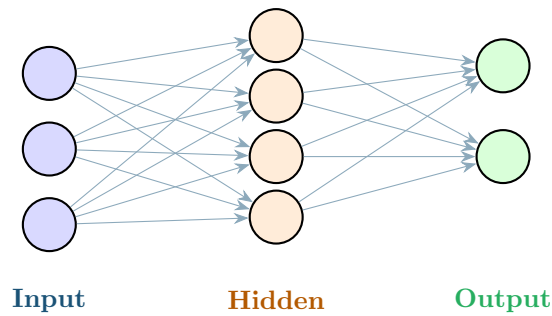
💡 Exploration vs. Exploitation

An RL agent must balance **exploitation** (choosing the best-known action) with **exploration** (trying new actions to discover better strategies) — commonly managed with an **ϵ -greedy** policy: act randomly with probability ϵ , otherwise choose the best-known action.

11 Neural Networks and Deep Learning

☰ Artificial Neural Network

An artificial neural network is a computing system of interconnected **neurons** organized in layers (input, hidden, output) that learn to map inputs to outputs by adjusting connection **weights**.



- **Activation Function:** Introduces non-linearity (e.g., ReLU, Sigmoid, Softmax) so networks can learn complex patterns.
- **Backpropagation:** The algorithm that computes gradients and updates weights to minimize error.
- **Deep Learning:** Neural networks with many hidden layers, powering modern image recognition (CNNs) and language models (Transformers).

11.1 The Perceptron: A Single Neuron

Perceptron

The simplest neural unit computes a weighted sum of inputs, adds a bias, and passes the result through an activation function:

$$z = \sum_{i=1}^n w_i x_i + b, \quad \hat{y} = f(z)$$

where f is an activation function such as a step function or sigmoid.

11.2 Convolutional Neural Networks (CNNs)

How CNNs Process Images

CNNs use small filters (kernels) that slide over the image to detect local patterns like edges and textures.

- **Convolution Layer:** Slides a filter over the input, computing dot products to produce a **feature map** that highlights specific patterns.
- **Pooling Layer (e.g., Max Pooling):** Downsamples feature maps, keeping the strongest signal in each region and reducing computation.
- **Fully Connected Layer:** Combines extracted features to make the final classification decision.

11.3 Recurrent Neural Networks (RNNs) and LSTMs

- **RNN:** Processes sequential data (text, time series) by feeding the output of each step back as input to the next, maintaining a “memory” of previous steps.
- **Vanishing Gradient Problem:** Standard RNNs struggle to learn long-range dependencies because gradients shrink over many steps.
- **LSTM (Long Short-Term Memory):** A specialized RNN with gates (input, forget, output) that control what information to keep or discard, better handling long sequences.

12 Natural Language Processing and Computer Vision

12.1 Natural Language Processing (NLP)

NLP

NLP enables computers to understand, interpret, and generate human language, powering chatbots, translation, sentiment analysis, and large language models like GPT.

- **Tokenization:** Splitting text into words or sub-words for processing.
- **Stopword Removal & Stemming/Lemmatization:** Removing common low-information words (“the”, “is”) and reducing words to their root form (“running” → “run”).
- **Part-of-Speech (POS) Tagging:** Labeling each word with its grammatical role (noun, verb, adjective).
- **N-grams:** Contiguous sequences of n words (e.g., bigrams, trigrams) used to capture local word context.
- **Word Embeddings:** Dense vector representations of words (e.g., Word2Vec, GloVe) that capture meaning, so similar words have similar vectors.
- **Attention and Transformers:** The attention mechanism lets a model weigh which words matter most to each other, forming the basis of Transformer models used in modern LLMs.
- **Sentiment Analysis:** Determining whether text expresses positive, negative, or neutral opinion.
- **Large Language Models (LLMs):** Transformer-based models trained on massive text corpora to generate and understand language (e.g., ChatGPT).

12.2 Computer Vision

Computer Vision

Computer vision enables machines to interpret and understand visual information from images and video, used in face recognition, medical imaging, and self-driving cars.

- **Image Classification:** Assigning a label to an entire image (e.g., cat vs. dog).
- **Object Detection:** Identifying and locating multiple objects within an image.
- **Image Segmentation:** Labeling every pixel of an image according to the object it belongs to.
- **Convolutional Neural Networks (CNNs):** The dominant deep learning architecture for vision tasks, using the convolution and pooling layers introduced in the Neural Networks section.

Worked Example: One Convolution Step

A 3×3 image region:

$$\begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{pmatrix}$$

convolved with a 3×3 filter of all 1's produces a single output value equal to the sum of

all 9 pixels: $1 + 2 + 0 + 0 + 1 + 2 + 1 + 0 + 1 = 8$. Sliding this filter across the full image produces a complete feature map.

13 Applications of AI

Domain	Example Applications
Healthcare	Disease diagnosis, medical imaging analysis, drug discovery.
Finance	Fraud detection, algorithmic trading, credit scoring.
Transportation	Self-driving cars, traffic prediction, route optimization.
Business & IT	Chatbots, recommendation systems, process automation.
Education	Personalized learning, automated grading, tutoring systems.

14 Ethical and Responsible AI

⚠ Key Concerns

As future IT professionals, you must consider the risks and responsibilities of deploying AI systems.

- **Bias and Fairness:** Models can inherit or amplify biases present in training data.
- **Privacy:** AI systems often require large amounts of personal data, raising data protection concerns.
- **Transparency and Explainability:** Complex models (“black boxes”) can be difficult to interpret or justify.
- **Accountability:** Determining responsibility when an AI system makes a harmful or incorrect decision.
- **Job Displacement:** Automation may replace certain human roles, requiring workforce reskilling.

💡 Responsible AI Principles

Fairness • Reliability and Safety • Privacy and Security • Inclusiveness • Transparency • Accountability.

15 Core Mathematical Foundations for AI

15.1 Linear Algebra Essentials

- **Vector:** Ordered list of numbers, e.g., $\mathbf{x} = [x_1, x_2, \dots, x_n]$.
- **Matrix:** 2D array used to represent datasets and transformations.
- **Dot Product:** $\mathbf{w} \cdot \mathbf{x} = \sum_i w_i x_i$, central in linear models and neural networks.

15.2 Probability Essentials

- **Conditional Probability:** $P(A|B) = \frac{P(A \cap B)}{P(B)}$.

- **Bayes' Theorem:**

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- Used in Naive Bayes classifiers, medical diagnosis, and uncertainty-aware AI decisions.

15.3 Optimization Essentials

- Training aims to minimize a loss function $L(\theta)$ over model parameters θ .
- Gradient descent update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

where η is the learning rate.

16 Exam-Oriented Worked Examples

Worked Example 1: Confusion Matrix

A classifier gives: $TP = 45, TN = 40, FP = 10, FN = 5$.

- Accuracy = $\frac{45+40}{100} = 0.85 = 85\%$
- Precision = $\frac{45}{45+10} = 0.818 = 81.8\%$
- Recall = $\frac{45}{45+5} = 0.90 = 90\%$
- F1 = $2 \cdot \frac{0.818 \cdot 0.90}{0.818 + 0.90} \approx 0.857 = 85.7\%$

Interpretation: strong recall means most positives are detected; moderate FP reduces precision.

Worked Example 2: Entropy and Information Gain

Consider dataset S with 8 samples: 4 positive and 4 negative.

$$H(S) = -\frac{4}{8} \log_2 \frac{4}{8} - \frac{4}{8} \log_2 \frac{4}{8} = 1$$

If attribute A splits into subsets with weighted entropy 0.5, then:

$$IG(S, A) = H(S) - 0.5 = 0.5$$

Higher information gain indicates a better split in decision trees.

Worked Example 3: One Gradient Descent Step

For loss $L(w) = (w - 3)^2$, gradient is $\nabla L = 2(w - 3)$. If $w_0 = 0$ and learning rate $\eta = 0.1$:

$$\nabla L(w_0) = 2(0 - 3) = -6$$

$$w_1 = w_0 - \eta \nabla L = 0 - 0.1(-6) = 0.6$$

The parameter moves toward the optimum $w = 3$.

</> Worked Example 4: Naive Bayes Classification

Predict whether to “Play” tennis given *Outlook* = *Sunny*, using training statistics $P(\text{Play}) = 0.6$, $P(\text{Sunny}|\text{Play}) = 0.33$, $P(\text{No Play}) = 0.4$, $P(\text{Sunny}|\text{No Play}) = 0.5$:

$$P(\text{Play}|\text{Sunny}) \propto 0.6 \times 0.33 = 0.198$$

$$P(\text{No Play}|\text{Sunny}) \propto 0.4 \times 0.5 = 0.200$$

Since $0.200 > 0.198$, Naive Bayes predicts **No Play** (the two scores are close, showing how sensitive the method is to the input statistics).

</> Worked Example 5: K-Nearest Neighbors

A new point has distances to 5 labeled neighbors: 1.2 (*A*), 2.0 (*A*), 2.5 (*B*), 3.0 (*B*), 3.1 (*B*). For $k = 3$, the three nearest neighbors are 1.2(*A*), 2.0(*A*), 2.5(*B*). Majority class = *A* (2 votes vs. 1), so the new point is classified as **A**.

</> Worked Example 6: One Iteration of K-Means

Points $\{2, 4, 10, 12\}$ with initial centroids $c_1 = 3, c_2 = 11$.

- Assign: $2 \rightarrow c_1, 4 \rightarrow c_1$ (closer to 3), $10 \rightarrow c_2, 12 \rightarrow c_2$ (closer to 11).
- Update centroids: $c_1 = \frac{2+4}{2} = 3, c_2 = \frac{10+12}{2} = 11$.

Centroids are unchanged, so the algorithm has already **converged**.

</> Worked Example 7: Genetic Algorithm — One Generation

Parent chromosomes (bit strings): $P_1 = 11001, P_2 = 10011$. Single-point crossover after position 2:

- Child $C_1 = \underbrace{11}_{\text{from } P_1} + \underbrace{011}_{\text{from } P_2} = 11011$
- Child $C_2 = \underbrace{10}_{\text{from } P_2} + \underbrace{001}_{\text{from } P_1} = 10001$

Mutation (flip last bit of C_1 with small probability): $11011 \rightarrow 11010$. These children form the next generation.

</> Worked Example 8: One Q-Learning Update

Given $Q(s, a) = 2$, reward $r = 1$, learning rate $\alpha = 0.5$, discount $\gamma = 0.9$, and $\max_{a'} Q(s', a') = 3$:

$$Q(s, a) \leftarrow 2 + 0.5[1 + 0.9(3) - 2] = 2 + 0.5(1.7) = 2.85$$

The Q-value increases from 2 to 2.85, reflecting a better-than-expected outcome.

17 Quick Revision Glossary

Term	One-Line Meaning
Agent	Entity that perceives via sensors and acts via actuators to achieve goals.
PEAS	Performance, Environment, Actuators, Sensors — framework for describing a task environment.
Heuristic $h(n)$	Estimated cost from node n to the goal, used to guide informed search.
Admissible Heuristic	A heuristic that never overestimates the true cost to the goal.
Minimax	Game-tree search where MAX maximizes and MIN minimizes the backed-up value.
Alpha-Beta Pruning	Skips branches in minimax that cannot affect the final decision.
CSP	Problem defined by variables, domains, and constraints; solved via backtracking.
Planning / STRIPS	Finding an action sequence to reach a goal using preconditions, add lists, and delete lists.
Iterative Deepening	Repeated DFS with increasing depth limits; combines BFS and DFS advantages.
Semantic Network	Graph representation of concepts and their relationships.
Forward Chaining	Data-driven inference: facts $\rightarrow$ rules $\rightarrow$ new facts $\rightarrow$ goal.
Backward Chaining	Goal-driven inference: start from the goal and verify supporting facts.
Fuzzy Logic	Reasoning with partial truth values (0 to 1) instead of strict true/false.
Overfitting	Model fits training data (including noise) too closely and generalizes poorly.
Underfitting	Model is too simple to capture the underlying pattern in the data.
Precision	$TP/(TP+FP)$ — how many predicted positives were actually correct.
Recall	$TP/(TP+FN)$ — how many actual positives were correctly found.
Entropy	Measure of impurity/disorder in a dataset; used to build decision trees.
Information Gain	Reduction in entropy achieved by splitting on a given attribute.
Gradient Descent	Iterative optimization that updates parameters opposite to the loss gradient.
Perceptron	Simplest neural unit computing a weighted sum plus bias through an activation function.
CNN	Neural network using convolution/pooling layers, dominant for image tasks.

Term	One-Line Meaning
RNN / LSTM	Neural networks for sequential data; LSTM adds gates to retain long-term memory.
Word Embedding	Dense vector representation of a word capturing semantic meaning.
K-Means	Unsupervised algorithm that groups data into k clusters via iterative centroid updates.
KNN	Classifies a point by majority vote among its k nearest labeled neighbors.
MDP	Formalizes RL via states, actions, transition probabilities, rewards, and a discount factor.
Policy	A strategy $\pi(s)$ mapping states to the actions an agent should take.
Q-Learning	Model-free RL algorithm that learns action values directly from experience.
Exploration vs. Exploitation	Trade-off between trying new actions and using the best-known action.

18 Question Bank Blueprint (80 Marks Support)

💡 Suggested 80 Marks Distribution

- **Section A:** MCQs and definitions (20 marks)
- **Section B:** Short conceptual questions (20 marks)
- **Section C:** Descriptive/theory questions (20 marks)
- **Section D:** Numerical/problem-solving questions (20 marks)

18.1 Important Theory Questions

- Differentiate symbolic AI, machine learning, and deep learning with examples.
- Explain and compare simple reflex, goal-based, and utility-based agents.
- Explain BFS, DFS, and A* with completeness and optimality.
- Describe PEAS with one real-world agent design.
- Explain the Minimax algorithm and how Alpha-Beta pruning improves it.
- Define a CSP and describe backtracking search with an example.
- Explain STRIPS-style planning and differentiate forward and backward planning.
- Differentiate forward chaining and backward chaining with an example each.
- Explain fuzzy logic and why it is useful for real-world control systems.
- Compare supervised, unsupervised, and reinforcement learning with examples.

- Define an MDP and explain the Q-learning update rule.
- Explain the exploration-exploitation trade-off in reinforcement learning.
- Explain overfitting, underfitting, and the bias-variance tradeoff.
- Describe how a CNN processes an image, mentioning convolution and pooling.
- Explain the difference between RNNs and LSTMs and why LSTMs help with long sequences.
- Discuss ethical issues in AI and practical mitigation methods.

18.2 Important Numerical/Analytical Questions

- Compute accuracy, precision, recall, and F1 from a confusion matrix.
- Solve one A* search expansion step given $g(n)$ and $h(n)$ values.
- Compute the backed-up values of a small minimax tree.
- Solve a 3-region map-coloring CSP by hand using backtracking.
- Construct a simple STRIPS plan given actions with preconditions and effects.
- Compute entropy and information gain for a small dataset split.
- Classify a test instance using Naive Bayes given simple probability tables.
- Classify a new point using KNN given distances to labeled neighbors.
- Perform one iteration of K-Means clustering given points and initial centroids.
- Perform one generation (crossover and mutation) of a genetic algorithm.
- Perform one Q-learning update given $Q(s, a)$, reward, learning rate, and discount factor.
- Perform one gradient descent update for a simple loss function.

18.3 Sample MCQs with Answer Key

1. Which search strategy is guaranteed to find the shallowest solution?
(a) DFS (b) BFS (c) Greedy Best-First (d) Hill Climbing
2. In Minimax, which player minimizes the backed-up value?
(a) MAX (b) MIN (c) Both (d) Neither
3. Alpha-beta pruning skips a branch when:
(a) $\alpha < \beta$ (b) $\alpha \geq \beta$ (c) $\alpha = 0$ (d) $\beta = 1$
4. Which metric is most reliable for a highly imbalanced dataset?
(a) Accuracy (b) F1-score (c) Raw error count (d) None
5. Which learning type uses labeled data?
(a) Unsupervised (b) Reinforcement (c) Supervised (d) Fuzzy
6. CNNs are primarily used for:
(a) Tabular data (b) Image data (c) Audio only (d) Text only
7. Backward chaining is best described as:
(a) Data-driven (b) Goal-driven (c) Random (d) Exhaustive only

8. In Q-learning, the term $\max_{a'} Q(s', a')$ represents:
(a) The immediate reward (b) The best estimated future value (c) The learning rate (d) The discount factor
9. STRIPS actions are defined using:
(a) Weights and biases (b) Preconditions and effects (c) Kernels and filters (d) Support vectors

💡 Answer Key

1–(b), 2–(b), 3–(b), 4–(b), 5–(c), 6–(b), 7–(b), 8–(b), 9–(b)

19 Summary

🔗 Expanded Key Takeaways

- AI combines reasoning, search, knowledge representation, and learning to build intelligent behavior.
- Agent design and search strategies (including adversarial search, CSPs, and planning) are central to classic AI problem solving.
- Knowledge representation, chaining, and uncertainty handling (Bayesian networks, fuzzy logic) let systems reason under real-world conditions.
- Machine learning, reinforcement learning, and deep learning provide data-driven and reward-driven intelligence, from classic algorithms (Naive Bayes, KNN, K-Means, Q-learning) to CNNs, RNNs, and Transformers.
- Evaluation metrics, probability, and basic mathematics are essential for exam success and practical model validation.
- Responsible AI principles must guide all stages of system design and deployment.